

Notebook: gradient_descent/general_gradient_descent_template.ipynb

[--- CODE CELL ---]

```
import random

def gradient_descent(derivative, learning_rate = 0.01, tolerance=1e-6, max_iters= 1000):
    #step 1: we start at a random point
    x = random.uniform(-5,5)
    print(f"Start x = {x:.6f}\n")
    for i in range(max_iters):
        slope = derivative(x)
        print(f"Iter {i:03d} | x = {x:.6f} | slope = {slope:.6f}")
        if abs(slope) < tolerance:
            break
        x = x - learning_rate * slope
    return x
```

[--- CODE CELL ---]

```
# function  $f(x) = 2(x + 5)$ 
def d1(x):
    return 2 * (x + 5)

xmin = gradient_descent(d1)
print("Minimum at X = ", xmin)
```

[--- CODE CELL ---]

```
def d2(x):
    return -2 * (x - 1)

xmin = gradient_descent(d2)
print("Minimum at X = ", xmin)
```

[--- CODE CELL ---]

```
import math

def d3(x):
    return 4 * x * math.cos(x) - 2 * (x**2) * math.sin(x) - 5

xmin = gradient_descent(d3, learning_rate = 0.001)
print("Minimum at X = ", xmin)
```

[--- CODE CELL ---]

```
def d4(x):
    return 3*(x ** 2) - 3

xmin = gradient_descent(d4, learning_rate = 0.001)
print("Minimum at X = ", xmin)
```

[--- CODE CELL ---]

[--- CODE CELL ---]

[--- CODE CELL ---]

```
# Two-parameter gradient descent (slope m and intercept b) for  $y = m*x + b$ 
import random
import numpy as np

def gradient_descent_2d(X, y, learning_rate=0.001, tolerance=1e-8, max_iters=20000):
    # start from small random values for slope (m) and intercept (b)
    m = random.uniform(-1, 1)
    b = random.uniform(-1, 1)
```

```

print(f"Start m = {m:.6f}, b = {b:.6f}\n")
n = len(X)
for i in range(max_iters):
    preds = m * X + b
    error = preds - y
    # gradients for MSE loss: d/dm and d/db
    grad_m = (2.0 / n) * np.sum(error * X)
    grad_b = (2.0 / n) * np.sum(error)
    # print progress occasionally
    if i % 500 == 0:
        loss = np.mean(error ** 2)
        print(f"Iter {i:04d} | m = {m:.6f} | b = {b:.6f} | grad_m = {grad_m:.6f} | grad_b = {grad_b:.6f} |
loss = {loss:.6f}")
    # stopping condition
    if abs(grad_m) < tolerance and abs(grad_b) < tolerance:
        print(f"Converged at iter {i}")
        break
    # update parameters
    m = m - learning_rate * grad_m
    b = b - learning_rate * grad_b
return m, b

# demo on a small synthetic dataset
rng = np.random.RandomState(0)
X = rng.rand(50) * 10
# true relationship y = 2.5*x - 1.0 + noise
y = 2.5 * X - 1.0 + rng.randn(50) * 0.5
m_found, b_found = gradient_descent_2d(X, y, learning_rate=0.001)
print('\nFound parameters: m =', round(m_found, 4), ', b =', round(b_found, 4))

```